TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

CHIA ĐỂ TRỊ

Từ cơ bản đến nâng cao

DIVIDE · CONQUER · COMBINE

TẬP 3 · HƯỚNG DẪN GIẢI



Lê Thành Nam

01 Chương 1. NỀN TẢNG CHIA ĐỂ TRỊ

02 Chương 2. TÌM KIẾM NHỊ PHÂN

03 Chương 3. CHIA ĐỂ TRỊ TRÊN DÃY, MẶT PHẲNG VÀ TẬP HỢP

04 Chương 4. CHIA ĐỂ TRỊ NÂNG CAO

Phân tích từ cách ngây thơ đến cách tối ưu · 42 bài

Dành cho học sinh THPT ôn luyện học sinh giỏi cấp tỉnh và tiếp cận cấp quốc gia

Tổng hợp và biên soạn: Lê Thành Nam

LỜI NÓI ĐẦU

Tập 3 trình bày lời giải cho 42 bài tập của Tập 2. Mỗi lời giải không chỉ đưa ra chương trình đạt 100 điểm, mà đi lại đúng con đường một học sinh giỏi suy nghĩ trong phòng thi: bắt đầu từ cách ngây thơ, đếm số phép tính để thấy bế tắc, rồi phá bế tắc bằng một quan sát hay một công cụ phù hợp.

Mọi chương trình trong sách đều đã được kiểm tra hai lần: đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu nhỏ ngẫu nhiên, và chấm trên bộ test chính thức với giới hạn 1 giây, 256 MB. Các con số trong bảng điểm cuối mỗi bài là điểm thực tế mà từng chương trình nhận được, không phải ước lượng. Một số bảng điểm có cả những chương trình sai: đó là những lỗi mà rất nhiều học sinh mắc phải, được giữ lại để em thấy chúng mất bao nhiêu điểm.

Lê Thành Nam

Lê Thành Nam

CẤU TRÚC MỘT LỜI GIẢI

Mục	Nội dung
1. Hiểu bài toán	Diễn đạt lại đề bằng ngôn ngữ của chia để trị, nhận ra mô hình quen thuộc ẩn sau câu chuyện.
2. Cách ngây thơ	Chương trình và độ phức tạp của cách làm đầu tiên ai cũng nghĩ ra.
3. Bế tắc	Vì sao cách hiện tại không vượt qua được subtask lớn; đôi khi kèm một lời giải trung gian hoặc một lời giải sai "trông như đúng".
4. Phá bế tắc	Cách chia, cách kết hợp, hình vẽ chạy tay, lập luận vì sao đúng.
5. Cách tối ưu	Chương trình hoàn chỉnh và độ phức tạp.
6. Bẫy thường gặp	Những lỗi làm mất điểm hay gặp nhất.
7. Bảng điểm	Điểm thực tế của từng cách làm trên bộ test. Dòng tô xanh là lời giải mẫu.

Lời khuyên: Đừng đọc lời giải như đọc truyện. Sau mục 3, hãy gấp sách và tự tìm cách phá bế tắc trong ít nhất mười lăm phút. Sau mục 5, hãy tự viết lại chương trình mà không nhìn sách, rồi tự sinh test để đối chiếu với cách ngây thơ của chính em.

Cách tự đối chiếu với lời giải vét cạn

Ba tệp là đủ: chương trình của em (sol.cpp), chương trình vét cạn (trau.cpp) và một chương trình sinh test ngẫu nhiên nhỏ (gen.cpp, nhận một hạt giống từ dòng lệnh). Đoạn lệnh dưới đây chạy hàng trăm test và dừng lại ở test đầu tiên hai chương trình cho kết quả khác nhau.

‹› Đối chiếu tự động (chạy trên Linux hoặc Git Bash)

```
for i in $(seq 1 500); do
  ./gen $i > test.inp
  ./sol < test.inp > sol.out
  ./trau < test.inp > trau.out
  if ! cmp -s sol.out trau.out; then echo "Khác nhau ở test $i"; break; fi
done
```

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CẤU TRÚC MỘT LỜI GIẢI **3**

CHƯƠNG 1. NỀN TẢNG CHIA ĐỂ TRỊ **6**

Bài 1.1 NHÂN BẢN 6
Bài 1.2 TỔNG LŨY THỪA 10
Bài 1.3 THÁP HÀ NỘI 14
Bài 2.1 NỔI BỌT 18
Bài 2.2 CẶP LỆCH 22
Bài 2.3 ĐOẠN TỔNG 27
Bài 3.1 TRUNG VỊ 32
Bài 3.2 HÀM ĐỆ QUY 36
Bài 3.3 ĐẾM SO SÁNH 40

CHƯƠNG 2. TÌM KIẾM NHỊ PHÂN **45**

Bài 4.1 TRA ĐIỂM 45
Bài 4.2 CẶP SỐ 48
Bài 4.3 HAI DÃY 51
Bài 5.1 MÁY IN 56
Bài 5.2 TỔNG THỨ K 60
Bài 5.3 LEO NÚI 64
Bài 6.1 CHIA BÁNH 69
Bài 6.2 TẬP TRUNG 73
Bài 6.3 TỈ LỆ 77

CHƯƠNG 3. CHIA ĐỂ TRỊ TRÊN DÃY, MẶT PHẲNG VÀ TẬP HỢP **81**

Bài 7.1 BIẾN ĐỘNG 81
Bài 7.2 ĐOẠN TRUNG BÌNH 86
Bài 7.3 LIỀN MẠCH 91
Bài 8.1 TRẠM XĂNG 96
Bài 8.2 ĐIỂM TRỘI 100
Bài 8.3 TAM GIÁC NHỎ NHẤT 104
Bài 9.1 MUA QUÀ 109
Bài 9.2 CÂN ĐĨA 113
Bài 9.3 ĐƯỜNG XOR 117
Bài 10.1 GẤP GIẤY 121
Bài 10.2 XÂU ABACABA 124
Bài 10.3 ĐƯỜNG HILBERT 128

CHƯƠNG 4. CHIA ĐỂ TRỊ NÂNG CAO **94**

Lê Thành Nam

Bài 11.1 NHỎ NHẤT 132
Bài 11.2 ĐOẠN TỐT NHẤT 136
Bài 11.3 DÃY NGOẶC 140
Bài 12.1 BƯU ĐIỆN 145
Bài 12.2 CÁP TREO 149
Bài 12.3 CHIA ĐOẠN 153
Bài 13.1 HAI TIÊU CHÍ 157
Bài 13.2 BẢN ĐỒ 161
Bài 13.3 RÚT THẺ 166
Bài 14.1 ĐỨNG K 170
Bài 14.2 ĐUA XE 175
Bài 14.3 TÔ ĐỎ 179

TỔNG KẾT: 42 BÀI, MƯỜI MỘT Ý TƯỞNG 184

LỜI KẾT 186

Lê Thành Nam

CHƯƠNG 1

NỀN TẢNG CHIA ĐỂ TRỊ

Mỗi bài trong tập này được phân tích theo cùng một trình tự: hiểu bài toán qua hình vẽ, cách ngây thơ (kèm chương trình và số điểm đạt được), chỗ bế tắc, các bước phá bế tắc, chương trình tối ưu, các bẫy thường gặp và bảng điểm của từng cách làm. Bảng điểm được đo thật bằng bộ test chấm của sách, với giới hạn 1 giây mỗi test.

BÀI 1.1 NHÂN BẢN

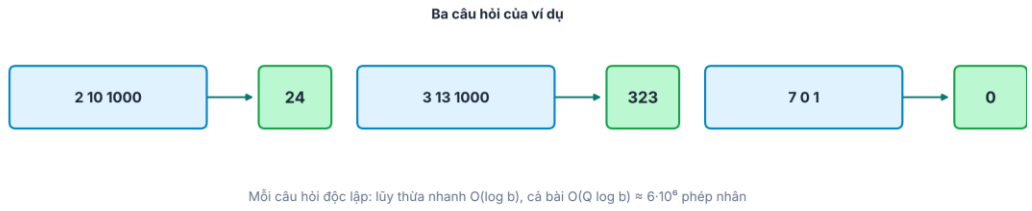
Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
NHANBAN.INP	NHANBAN.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Bài 1

Lê Thành Nam

1. Hiểu bài toán

Số tế bào sau b phút là a^b. Bài toán chính là bài LUYTHUA của Bài 1, nhưng phải trả lời tới 10^5 câu hỏi, mỗi câu với số mũ tới 10^{18}. Hai chi tiết cần để ý ngay từ đầu: a có thể lớn hơn m (phải lấy dư trước khi nhân), và m có thể bằng 1 (khi đó mọi đáp án là 0, kể cả a^0).



Hình 1.1.1. *Ba câu hỏi của ví dụ: mỗi câu được trả lời độc lập bằng lũy thừa nhanh*

2. Cách ngây thơ: Nhân lần lượt

Với mỗi câu hỏi, bắt đầu từ 1 và nhân với a đúng b lần, lấy dư sau mỗi phép nhân.

Cách ngây thơ: nhân lần lượt, O(Q · b)

```cpp
#include <cstdio>
#include <iostream>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("NHANBAN.INP", "r")) {
        freopen("NHANBAN.INP", "r", stdin);
        freopen("NHANBAN.OUT", "w", stdout);
    }
    int q;
    cin >> q;
    while (q--) {
        long long a, b, mod;
        cin >> a >> b >> mod;
        a %= mod;
        long long res = 1 % mod;
        for (long long step = 1; step <= b; step++)
            res = res * a % mod;
        cout << res << '\n';
    }
    return 0;
}
```

Độ phức tạp: O(Q · b). Subtask 1: $10 \cdot 10^6 = 10^7$ phép nhân. Subtask 2: $10^5 \cdot 1000 = 10^8$ phép nhân, vẫn kịp. Subtask 3: b tới 10^{18}, không bao giờ xong.

Một cách "nhanh" nhưng sai: hàm pow của cmath

Nhiều bạn viết fmod(pow(a, b), m). Hàm pow tính trên số thực double, chỉ giữ khoảng 16 chữ số có nghĩa; 3^{13} đã có 7 chữ số, 2^{64} có 19 chữ số, và với b lớn, kết quả là vô cùng (inf). Chương trình này **sai ở mọi subtask** (xem bảng điểm).

3. Bế tắc

- **Subtask 1, 2** ($b \le 10^6$ và $b \le 1000$): nhân lần lượt chạy kịp: **60 điểm**.
- **Subtask 3** (b tới 10^{18}): 10^5 câu hỏi × 10^{18} phép nhân: **0 điểm**.

> **■ ĐIỂM BẾ TẮC**
>
> Mỗi phép nhân chỉ tăng số mũ thêm 1. Cần một cách làm số mũ **gấp đôi** sau mỗi bước.

4. Phá bế tắc: Lũy thừa nhanh

Dùng đúng công thức của Bài 1: $a^b = h \cdot h$ (nhân thêm a nếu b lẻ), với $h = a^{\lfloor b/2 \rfloor}$. Mỗi câu hỏi tốn không quá 60 lời gọi.

Chạy tay câu hỏi thứ nhất (a = 2, b = 10, m = 1000). Số mũ đi xuống 10 → 5 → 2 → 1 → 0, kết quả đi lên:

Lê Thành Nam

Lời gọi	b chẵn / lẻ	h	Kết hợp	Kết quả
power(2, 0)	điểm dừng			**1**
power(2, 1)	lẻ	1	1 · 1 · 2	**2**
power(2, 2)	chẵn	2	2 · 2	**4**
power(2, 5)	lẻ	4	4 · 4 · 2	**32**
power(2, 10)	chẵn	32	32 · 32 = 1024 → 24	**24**

Câu hỏi thứ ba (7 0 1): lời gọi power(7, 0) trả về 1 % 1 = 0, đúng như ví dụ.

5. Cách tối ưu

NHANBAN.CPP: Lũy thừa nhanh cho từng câu hỏi, O(Q log b)

```cpp
#include <cstdio>
#include <iostream>
using namespace std;

long long power(long long a, long long b, long long mod) {
    if (b == 0) return 1 % mod;
    long long half = power(a, b / 2, mod);
    long long res = half * half % mod;
    if (b % 2 == 1) res = res * a % mod;
    return res;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("NHANBAN.INP", "r")) {
        freopen("NHANBAN.INP", "r", stdin);
        freopen("NHANBAN.OUT", "w", stdout);
    }
    int q;
    cin >> q;
    while (q--) {
        long long a, b, mod;
        cin >> a >> b >> mod;
        cout << power(a % mod, b, mod) << '\n';
    }
    return 0;
}
```

Lê Thành Nam

Độ phức tạp: mỗi câu hỏi O(log b) ≈ 60 lời gọi, tổng cộng khoảng $10^5 \cdot 60 \cdot 2 \approx 1{,}2 \cdot 10^7$ phép nhân. Thời gian chạy khoảng 0,1 giây, phần lớn là để đọc dữ liệu. Vì phải in tới 10^5 dòng, nên dùng '\n' thay cho endl (endl đẩy dữ liệu ra tệp sau mỗi dòng, rất chậm).

6. Bẫy thường gặp

- **Trả về 1 khi b = 0:** sai với m = 1. Phải trả về 1 % mod.
- **Quên lấy dư a cho m trước:** a = 10^9 và m nhỏ thì vẫn không tràn, nhưng đây là thói quen nên giữ.
- **Đọc b vào biến int:** b tới 10^{18} cần long long.
- **Dùng pow của cmath:** sai số học, 0 điểm.

- **Gọi đệ quy hai lần** power(a, b/2) · power(a, b/2): trở lại O(b), mất subtask 3.

7. Bảng điểm theo thuật toán

Thuật toán	ST1 (30)	ST2 (30)	ST3 (40)	Tổng
Dùng hàm pow của C++ (sai)	0	0	0	**0**
Nhân lần lượt b lần, O(Q·b)	30	30	0	**60**
Lũy thừa nhanh, O(Q·log b)	**30**	**30**	**40**	100

Lê Thành Nam